

Tätigkeitsbericht

Projekt 3 · Vom Excel-Lagersystem zur Cloud-Anwendung

Ziel: Das Excel/VBA-Lagerbestand-System aus Projekt 2 sollte, wie in Punkt 3 der ursprünglichen Aufgabenliste vorgesehen, in eine eigenständige, mehrbenutzerfähige Web-Anwendung überführt werden – vom lokalen Einzelplatzbetrieb bis zu einer privat erreichbaren Cloud-Version.

Methodik: Dieser Bericht listet alle 75 Arbeitsschritte aus dem Werkstattlogbuch zu Projekt 3 der Reihe nach, aufgeteilt auf 9 Arbeitstage à ca. 9 Stunden (letzter Tag kürzer). Jede Aufgabe trägt eine Kategorie (Dokumentation/Fehler/Korrektur-Lösung/Problem/Optimierung/Zusatzaufgabe) sowie ein Herkunfts-Etikett: „belegt“ (steht so in den Original-Quelldateien) oder „kreativ ergänzt“ (plausibel ergänzt, um den Ablauf realistisch zu zeigen). Arbeitszeiten sind selbst nach Aufwand geschätzt, nicht aus einer Zeiterfassung übernommen.

Zeitraum: bis 16.08.2026 (Enddatum aus den Quellen; Startdatum dort nicht dokumentiert)

Gesamtstunden: 77,5 h über 9 Arbeitstage

Quellen: projekte/projekt-3/index.html bis 13-fazit.html ·

quellen/dateien/05_lagersystem-projektbericht.pdf ·

quellen/dateien/06_lagersystem-projektbericht-detailliert.pdf

75	9	77,5 h	4	6	4
Aufgaben gesamt	Arbeitstage	Gesamtzeit	Fehler	Probleme	Optimierungen

Übersicht der Arbeitstage

Tag	Aufgaben	Anzahl	Stunden
Tag 1	Aufgabe 1-10	10	9,25 h
Tag 2	Aufgabe 11-17	7	9 h
Tag 3	Aufgabe 18-26	9	9,5 h
Tag 4	Aufgabe 27-34	8	9,25 h
Tag 5	Aufgabe 35-43	9	8,5 h
Tag 6	Aufgabe 44-50	7	8 h
Tag 7	Aufgabe 51-59	9	9,5 h
Tag 8	Aufgabe 60-69	10	9,25 h
Tag 9	Aufgabe 70-75	6	5,25 h

Tag	Aufgaben	Anzahl	Stunden
Gesamt		75	77,5 h

Legende Kategorien: Dokumentation (regulärer Arbeitsschritt) · Fehler (eigener Fehler, korrigiert)
 · Korrektur/Lösung (behebt Fehler oder Problem) · Problem (externes Hindernis) · Optimierung
 (eigenständig gefundene Verbesserung) · Zusatzaufgabe (ergänzende, nicht zwingend nötige
 Leistung)

Aufgabe 1: Ausgangslage: Projekt 2 abgeschlossen, Punkt 3 der Aufgabenliste steht noch aus

Dokumentation

belegt

0 h

- Das Excel/VBA-Lagerbestand-System aus Projekt 2 läuft produktiv und zuverlässig auf einem einzelnen Rechner.
- Punkt 3 der ursprünglichen Aufgabenliste ist noch offen: das Excel-Lagersystem „in eine Webseite umwandeln“.
- Noch keine eigene Leistung an Projekt 3: kein Code analysiert, keine Architekturentscheidung getroffen.

Aufgabe 2: Kickoff: Auftrag zur Umwandlung in eine Webseite angenommen

Dokumentation

belegt

0,5 h

- Auftrag eingeordnet: „Lager System V3.3.xlsm“ soll als eigenständige, mehrbenutzerfähige Anwendung neu aufgebaut werden.
- Vorgabe von Anfang an: Lösung so einfach wie möglich halten, Projekt Schritt für Schritt gemeinsam entwickeln.

Aufgabe 3: Tablet-Modus und Entwickler-Modus analysiert

Dokumentation

belegt

1 h

- Modul A3_Tablet_Modus / Sub APPModus_Umschalten durchgearbeitet: blendet Menüband, Bearbeitungsleiste und Blattregister aus.
- Modul A4_AnzichtBereich: zoomt jedes Blatt automatisch auf seinen Datenbereich.
- Workbook_Open nachvollzogen: entsperrt Blätter, gibt gezielt Zellbereiche frei, sperrt danach wieder komplett.

Aufgabe 4: Die sechs Tabellenblätter durchgearbeitet

Dokumentation

belegt

1 h

- Dashboard, Artikel Liste, Reste, Verlauf, Artikel Gruppe, Einstellung – Zweck und Zellschutz je Blatt dokumentiert.
- Auffällig: „Artikel Gruppe“ bewusst nur A-G freigegeben, „Artikel Liste“ dagegen A-Q.

Aufgabe 5: Die vier UserForms analysiert

Dokumentation

belegt

1,25 h

- frmArtikelDetails, frmArtikelGruppe, frmBarcodeEingabe und frmMaterialEntnahme im Detail durchgearbeitet.
- Kernstück erkannt: frmMaterialEntnahme berechnet die Restlänge live und fragt unter 1000 mm per Ja/Nein nach, ob Schrott.

Aufgabe 6: Die 24 VBA-Module A-H systematisch durchgearbeitet

Dokumentation	belegt	2,5 h
---------------	--------	-------

- Acht Themenbereiche durchgearbeitet: A Grundgerüst, B Export, C E-Mail, D Hilfsfunktionen, E Profil-/Material-Nachschlage, F Artikelgruppen, G Artikel-Stück (Kernlogik), H Dashboard-Suche.
- Herzstück identifiziert: G7_MaterialEntnahme_Logik mit ProzessMaterialEntnahme.
- Auffällig in H1_Suchen_Funktionen: Platzhalter- und auskommentierter Code für eine nie fertiggestellte Zusatzfunktion.

Aufgabe 7: Datenmodell der Excel-Tabellen rekonstruiert

Dokumentation	belegt	1 h
---------------	--------	-----

- Aus G1_ArtikelStueck_Type und den InTabelleHinzufuegen_AS-Zuweisungen den 14-spaltigen Aufbau von Artikel_Liste/Reste/Verlauf rekonstruiert.
- Artikel_Gruppe als schmalere 5-Spalten-Tabelle erkannt (Barcode, Bezeichnung, Profil, Maß, Material).

Aufgabe 8: Sechs strukturelle Schwachpunkte dokumentiert

Dokumentation	belegt	1 h
---------------	--------	-----

- Kein Mehrbenutzerbetrieb, durchgängige Mac/Windows-Plattformweichen, Gmail nie wirklich unterstützt.
- Unvollständiger Code in der Dashboard-Suche, Blattschutz mit Klartext-Passwort im Quellcode.
- Fehlende Nachvollziehbarkeit von Löschvorgängen (Del_ArtikelStueck sofort und endgültig).
- Diese Beobachtungen fließen direkt in die Anforderungen an die Neuentwicklung ein.

Aufgabe 9: Alternativen zur Weiternutzung von Excel diskutiert

Dokumentation	belegt	0,5 h
---------------	--------	-------

- Verschiedene Optionen abgewogen, bevor eine Richtung festgelegt wurde.

Aufgabe 10: Entscheidung: Python + Streamlit + SQLite, so einfach wie möglich

Dokumentation	belegt	0,5 h
---------------	--------	-------

- Bewusst gegen Excel entschieden – Web-Anwendung mit robusterer Datenbank im Hintergrund.
- Ausdrückliche Vorgabe: Einfachheit als Leitprinzip, bewusst gegen zusätzliche Komplexität solange nicht nötig.

Aufgabe 11: Drei-Schichten-Architektur festgelegt

Dokumentation

belegt

0,75 h

- app.py (Oberfläche), logic.py (Fachlogik), db.py (Datenzugriff), schema.sql (Datenbankschema) klar getrennt festgelegt.
- Regel definiert: app.py ruft nur logic.py auf, diese liest/schreibt nur über db.py – nie direktes SQL an der Datenbank vorbei.

Aufgabe 12: Datenbankschema entworfen

Dokumentation

belegt

2 h

- Stammdaten Lager, Typ, Einheit, Profil_Abk, Material_Gruppe angelegt.
- Artikel_Gruppe als Artikel-Stammsatz sowie die drei identisch aufgebauten Tabellen Artikel_Liste, Reste, Verlauf – bewusst nah an der Excel-Struktur.
- Barcode-Schema übernommen: Haupt-Barcode je Artikelgruppe, Stück-Barcode je physischem Stück.

Aufgabe 13: Falscher Schritt: Barcode-Schema zunächst ohne Trennzeichen entworfen

Fehler

kreativ ergänzt

0,5 h

- Im ersten Entwurf des Barcode-Schemas wurde der Stück-Barcode direkt an den Haupt-Barcode angehängt (z. B. „FLS3700012“ statt „FLS370001-2“) – beim Testen ließ sich damit nicht mehr eindeutig zwischen Haupt- und Stück-Nummer unterscheiden.

Aufgabe 14: Korrektur: Bindestrich als Trennzeichen ergänzt

Korrektur/Lösung

kreativ ergänzt

0,5 h

- Schema auf „Haupt-Barcode-Stücknummer“ (z. B. „FLS370001-2“) umgestellt – damit lässt sich ein gescannter Gruppen-Code eindeutig von einem gescannten Stück-Code unterscheiden.

Aufgabe 15: Materialentnahme-Logik nach Python portiert

Dokumentation

belegt

2,5 h

- G7_MaterialEntnahme_Logik fast 1:1 in logic.process_material_entnahme() übertragen.
- Reihenfolge beibehalten: Verlauf-Eintrag vor der Reduktion, dann Reduktion/Löschung des Ursprungsstücks, dann bedingtes Einbuchen des Reststücks.

Aufgabe 16: Einstellbare Schrott-/Rest-Schwelle ergänzt

Dokumentation

belegt

0,75 h

- Im Original war „1000 mm“ fest im Code verankert – neu über eine Konfigurationstabelle einstellbar gemacht.

Aufgabe 17: Mehrbenutzerbetrieb umgesetzt

Dokumentation

belegt

2 h

- ___ SQLite-Datenbank im WAL-Modus betrieben – lesende und schreibende Zugriffe blockieren sich nicht mehr gegenseitig.
- ___ Barcode-Vergabe zusätzlich über einen anwendungsinternen threading.Lock abgesichert.
- ___ Dieser gesamte Themenbereich existierte im Original-VBA-System überhaupt nicht.

Aufgabe 18: Lasttest zur Verifikation durchgeführt

Dokumentation

belegt

1 h

- ___ 100 gleichzeitige Schreibzugriffe, 5 parallele simulierte Nutzer.
- ___ Ergebnis: 0 Konflikte, keine doppelt vergebenen Barcodes.

Aufgabe 19: Export & Kommunikation bewusst vereinfacht

Dokumentation

belegt

1 h

- ___ Statt plattformabhängiger Ordnerdialoge: Streamlits eingebauter Datei-Download.
- ___ Statt Outlook-COM/AppleScript: einfacher mailto-Link, bewusst ohne SMTP-Zugangsdaten – funktioniert dadurch auch mit Gmail zuverlässig.

Aufgabe 20: VBA→Python-Funktionen tabellarisch gegenübergestellt

Dokumentation

belegt

1 h

- ___ Elf zentrale VBA-Funktionen ihren Python-Pendants zugeordnet.
- ___ Festgestellt: SheetsSperrungen/SheetsEntsperren entfällt komplett – kein Blattschutz-Konzept mehr nötig.

Aufgabe 21: Vier Fotos handschriftlicher Bestandslisten gesichtet

Dokumentation

belegt

0,5 h

- ___ Die realen Lagerbestände lagen zunächst nur als vier Fotos handbeschriebener Listen vor – als Ausgangsmaterial für die Digitalisierung.

Aufgabe 22: Kontroll-Tabelle statt direkter Übernahme erstellt

Dokumentation

belegt

2 h

- ___ Um Übertragungsfehler zu vermeiden, keine direkte Übernahme in die Produktivdatenbank.
- ___ Stattdessen „Bestandsaufnahme_Kontrolle.xlsx“ erstellt: alle erkannten Werte samt unsicherer/unleserlicher Stellen aufgelistet.

Aufgabe 23: Kontroll-Tabelle durch Auftraggeber bestätigt

Dokumentation

belegt

1,5 h

- ___ Vollständig durchgesehen und korrigiert, inklusive expliziter Kennzeichnung unbekannter Materialangaben mit „---“.

Aufgabe 24: Import per Skript in die echte Datenbank

Dokumentation

belegt

1 h

- ___ Erst auf Basis der final bestätigten Tabelle: import_bestand.py ausgeführt.
- ___ 86 Artikel-Datensätze über 50 Artikelgruppen hinweg importiert – bildet ab hier durchgehend die reale Produktivgrundlage.

Aufgabe 25: PDF-Download-Button zu jeder Tabellenansicht ergänzt

Dokumentation	belegt	1 h
---------------	--------	-----

— Neben dem eingebauten CSV-Download, direkt neben dem nativen Symbol-Menü platziert.

Aufgabe 26: Optimierung: PDF-Button-Position bei gefilterten Tabellen vereinheitlicht

Optimierung	belegt	0,5 h
-------------	--------	-------

— Position bei gefilterten/gesuchten Tabellen nachträglich vereinheitlicht: der PDF-Button steht seither immer oben neben der Ergebnis-Überschrift, nicht mehr darunter.

Aufgabe 27: Modale Dialoge statt eigener Unterseiten eingeführt

Dokumentation

belegt

2 h

- ___ Hinzufügen, Material entnehmen, Bearbeiten/Löschen, Artikelgruppe anlegen, Materialgruppe anlegen von eigenen Unterseiten auf st.dialog-Popups umgestellt.
- ___ Fenster öffnen sich per Knopfdruck, ohne die Seite zu wechseln.

Aufgabe 28: Dialog-Layout vereinheitlicht

Dokumentation

belegt

1 h

- ___ Fenster vertikal mittig zentriert, Eingabefelder konsequent untereinander statt nebeneinander.
- ___ Bestätigungs-Buttons einheitlich rechts platziert (Löschen links, Speichern/Hinzufügen rechts).

Aufgabe 29: Optimierung: Sechs Buttons zu drei zusammengeführt

Optimierung

belegt

1,5 h

- ___ Ursprünglich getrennte Buttons für „Artikel“ und „Reste“ zusammengeführt: die eingegebene Länge entscheidet seither automatisch, ob ein Stück als voller Artikel oder als Rest gespeichert wird – mit Live-Hinweistext und manueller Überschreibungs-Möglichkeit.

Aufgabe 30: Tabellen ohne inneren Scrollbalken umgestellt

Dokumentation

belegt

0,5 h

- ___ Tabellen zeigen seither ihre volle Höhe – die Seite selbst scrollt.

Aufgabe 31: Tipp-/scanfähige Kombobox mit „Erweiterten Filtern“ ergänzt

Dokumentation

belegt

1,5 h

- ___ Einfache Textsuche zu einer tipp- und scanfähigen Dropdown-Kombobox ausgebaut.
- ___ Abschnitt „Erweiterte Filter“ mit Mehrfachkriterien ergänzt: Profil, Barcode, Bezeichnung, Lagerort, Material, Maß, Länge von/bis.

Aufgabe 32: Optimierung: Schrott-/Rest-Grenze vereinheitlicht

Optimierung

belegt

1 h

- ___ Ursprünglich zwei getrennte, fest im Code verankerte Schwellenwerte auf eine einzige, über eine Konfigurationstabelle einstellbare Einstellung vereinheitlicht.

Aufgabe 33: Optimierung: Schnellzugriff zentral in die Seitenleiste verschoben

Optimierung

belegt

0,75 h

- ___ Schnellzugriff-Buttons waren zunächst auf allen Einzelseiten dupliziert – zentral in die Seitenleiste verschoben, damit sie von jeder Seite verfügbar sind, ohne redundante Pflege.

Aufgabe 34: Weitere Layout-Feinjustierungen

Dokumentation

belegt

1 h

- Überschriftengrößen, Spaltenanzahl in Formularen, einheitliche Button-Breiten.
- Eigene Filterleisten direkt auf „Artikel Liste“ und „Reste“ ergänzt.

Aufgabe 35: Problem: Risiko erkannt - „Löschen“ entfernte Artikel sofort & endgültig

Problem

belegt

0,5 h

- Sowohl im VBA-Original als auch in der ersten Version der neuen Anwendung entfernte der „Löschen“-Knopf einen Artikel nach nur einer Ja/Nein-Abfrage sofort und endgültig – bei einer Verwechslung beim Scannen wäre das ein potenzieller Datenverlust ohne jede Rückgängig-Möglichkeit gewesen.

Aufgabe 36: Lösung: Zwei Absicherungsstufen umgesetzt

Korrektur/Lösung

belegt

2 h

- Vor jedem Löschvorgang erscheint seither eine explizite Sicherheitsabfrage.
- Ein gelöscht Stück landet zusätzlich nicht mehr im Nichts, sondern in einer neuen Papierkorb-Tabelle samt Löschzeitpunkt und ursprünglicher Quelltable.

Aufgabe 37: Wiederherstellen oder endgültig löschen umgesetzt

Dokumentation

belegt

1 h

- Aus dem Papierkorb mit einem Klick wiederherstellbar (schlägt fehlerhaft ab, falls der Barcode inzwischen erneut vergeben wurde).
- Nach einer zweiten, eigenen Sicherheitsabfrage endgültig und unwiderruflich löschar.

Aufgabe 38: Backup-Skript mit SQLite VACUUM INTO umgesetzt

Dokumentation

belegt

1 h

- Sichert die Datenbank in eine eigenständige, in sich konsistente Kopie.

Aufgabe 39: Fehler: Backup lief zunächst nur beim Start der Anwendung

Fehler

belegt

0,5 h

- Bei einem durchgehend laufenden Hintergrunddienst wäre das Backup dadurch nie aktualisiert worden – die Datei blieb praktisch immer auf dem Stand des letzten Neustarts.

Aufgabe 40: Korrektur: Zusätzlicher täglicher Backup-Zeitplan ergänzt

Korrektur/Lösung

belegt

1 h

- Zusätzlich ein täglich zu fester Uhrzeit ausgelöster Backup-Zeitplan ergänzt, unabhängig von Neustarts. Es werden automatisch die letzten 30 Tage aufbewahrt.

Aufgabe 41: macOS-LaunchAgent für automatischen Hintergrunddienst eingerichtet

Dokumentation

belegt

1,5 h

- Anwendung startet seither automatisch beim Anmelden am Rechner.
- Startet sich bei einem Absturz selbstständig neu.

Aufgabe 42: Absturz-Neustart gezielt verifiziert

Dokumentation	belegt	0,5 h
---------------	--------	-------

___ Prozess im laufenden Betrieb gezielt beendet, um den automatischen Neustart zu überprüfen.

Aufgabe 43: Problem: iCloud Drive verweigert Hintergrunddiensten den Dateizugriff

Problem	belegt	0,5 h
---------	--------	-------

___ Der Projektordner lag ursprünglich in iCloud Drive – macOS verweigert Hintergrunddiensten dort systembedingt den Dateizugriff. Kein eigener Fehler, sondern eine systembedingte Einschränkung von macOS.

Aufgabe 44: Lösung: Projekt in lokalen Ordner umgezogen

Korrektur/Lösung

belegt

1,5 h

- Nach Rücksprache das gesamte Projekt in einen lokalen, nicht synchronisierten Ordner verschoben (die ursprüngliche Kopie blieb – nur umbenannt – als Sicherheitsnetz erhalten), die virtuelle Python-Umgebung neu aufgebaut und der Hintergrunddienst umkonfiguriert.

Aufgabe 45: Kurzanleitung für den Vorgesetzten erstellt

Zusatzaufgabe

belegt

1 h

- Ergänzend zur technischen README: Öffnen der App, Übersicht der Seiten, wichtigste Aktionen im Schnellzugriff, Vorgehen bei versehentlichem Löschen.

Aufgabe 46: Verknüpfungen für macOS, Windows, iOS/Android erstellt

Zusatzaufgabe

belegt

1 h

- .webloc-Datei für macOS, .url-Datei für Windows – beide direkt auf die Netzwerkadresse zeigend, kein Installationsschritt nötig.
- Für iOS/Android die Vorgehensweise „Zum Home-Bildschirm hinzufügen“ dokumentiert.

Aufgabe 47: Vollständiges Windows-Setup-Paket vorbereitet

Zusatzaufgabe

belegt

2,5 h

- Plattformunabhängiges Backup-Skript in Python statt Bash.
- Start-Skript per Windows-Aufgabenplanung, unsichtbar über einen VBS-Wrapper gestartet, plus manueller Doppelklick-Fallback.
- Ausführliche Schritt-für-Schritt-Anleitung – Rückfalloption, falls der Betrieb auf einen Firmenrechner umzieht.

Aufgabe 48: Problem: struktureller Schwachpunkt – System an eine einzelne Person gebunden

Problem

belegt

0,5 h

- Bei der Planung der Übergabe erkennt: Der Praktikant, auf dessen Rechner die Anwendung lief, würde das Unternehmen irgendwann verlassen – ein System, das an eine einzelne Person gebunden ist, wäre dann nicht mehr betriebsfähig. Kein eigener Fehler, sondern ein struktureller Umstand der Ausgangslage.

Aufgabe 49: Zwei Wege gegenübergestellt

Dokumentation

belegt

1 h

- Firmenrechner: kostenlos, kein zusätzliches Sicherheitsrisiko, aber weiterhin auf das Büro-Netzwerk beschränkt.
- Echtes Cloud-Hosting: von überall erreichbar, aber echte Kosten, notwendiger Zugriffsschutz und größerer technischer Umbau.

Aufgabe 50: Lösung: Entscheidung für die Cloud getroffen

Korrektur/Lösung

belegt

0,5 h

- GitHub für die Quellcodeverwaltung, Neon als kostenlose gehostete Postgres-Datenbank, Streamlit Community Cloud als Ausführungsumgebung mit privatem, einladungsbasiertem Zugriff – alle drei Dienste in der kostenlosen Stufe ausreichend.

Aufgabe 51: Zwei-Wege-Datenzugriffsschicht in db.py umgesetzt

Dokumentation

belegt

2 h

- ___ Ist DATABASE_URL konfiguriert, verbindet sich die Anwendung mit Postgres, sonst bleibt sie beim lokalen SQLite-Betrieb.
- ___ Exakt derselbe Anwendungscode läuft wahlweise lokal oder in der Cloud – ohne Verzweigungen im restlichen Programm.

Aufgabe 52: Postgres-Schema und Migrationsskript geschrieben

Dokumentation

belegt

2 h

- ___ schema_postgres.sql: syntaktisch angepasstes, aber inhaltlich identisches Schema.
- ___ migrate_sqlite_to_postgres.py: überträgt bestehende Daten vollständig und wiederholbar in die Cloud-Datenbank.

Aufgabe 53: Lokale Postgres-Testinstanz aufgesetzt

Dokumentation

belegt

1 h

- ___ Eigens installiert, mit einer Kopie der echten 86 Datensätze befüllt, um systematisch gegen echtes Postgres-Verhalten zu testen.

Aufgabe 54: Problem 1/4: Groß-/Kleinschreibung – Postgres faltet Bezeichner auf Kleinschreibung

Problem

belegt

0,75 h

- ___ Anders als SQLite faltet Postgres unquotierte Tabellen- und Spaltennamen automatisch auf Kleinschreibung – „LaengeMM“ kommt als „laengemm“ zurück. Ein Unterschied im Datenbankverhalten, kein eigener Programmierfehler.

Aufgabe 55: Lösung 1/4: _Zeile-Klasse gebaut

Korrektur/Lösung

belegt

1,5 h

- ___ Eigenes Zeilen-Objekt sorgt seither dafür, dass Ergebniszeilen unabhängig vom Datenbanktyp exakt die im Schema definierte Schreibweise zeigen – Schlüssel werden schon beim Erstellen umbenannt, nicht erst bei individuellem Feldzugriff.

Aufgabe 56: Problem 2/4: Cursor-Typ-Konflikt mit pandas

Problem

belegt

0,5 h

- ___ Der für den eigenen dict-artigen Zeilenzugriff gesetzte Cursor-Typ kollidierte mit der Art, wie die Bibliothek pandas Tabellen aus einer Datenbankverbindung einliest.

Aufgabe 57: Lösung 2/4: Getrennte Cursor-Konfigurationen

Korrektur/Lösung

belegt

0,75 h

- ___ Für beide Anwendungsfälle (eigener dict-Zugriff vs. pandas-Einlesen) jeweils eine eigene Cursor-Konfiguration angelegt.

Aufgabe 58: Falscher Schritt 3/4: Alias-Kollision in einer Abfrage

Fehler

belegt

0,5 h

- ___ Ein in einer Abfrage selbst gewählter Kurzname (Alias) geriet in Konflikt mit einer echten Spalte gleichen Namens in einer anderen Tabelle und lieferte dadurch falsche Werte zurück – eine eigene Namenswahl beim Schreiben der Abfrage.

Aufgabe 59: Korrektur 3/4: Abfrage-Alias eindeutig umbenannt

Korrektur/Lösung

belegt

0,5 h

- ___ Kurzname eindeutig umbenannt, sodass keine Kollision mit einer echten Spalte mehr auftreten kann.

Aufgabe 60: Problem 4/4: Sequenz-Fortsetzung nach Datenübertragung nicht synchron

Problem

belegt

0,5 h

- Nach dem Übertragen bestehender Datensätze mit bereits vergebenen IDs blieb der interne Postgres-Zähler auf dem alten Stand – ein Verhalten von Postgres beim Einspielen fremder IDs, kein eigener Fehler.

Aufgabe 61: Lösung 4/4: Sequenz manuell auf höchsten Wert gesetzt

Korrektur/Lösung

belegt

0,5 h

- Interner Postgres-Zähler nach jeder Übertragung per setval() manuell auf den höchsten übertragenen Wert gesetzt, um spätere ID-Kollisionen zu vermeiden.

Aufgabe 62: Jede Korrektur lokal gegen echte Daten verifiziert

Dokumentation

belegt

1 h

- Alle vier Korrekturen vor der Veröffentlichung lokal gegen die Postgres-Testinstanz mit einer Kopie der echten 86 Datensätze nachgestellt.
- Durch einen vollständigen Klick-Test aller Seiten und Dialoge verifiziert, bevor live geschaltet wurde.

Aufgabe 63: Privates GitHub-Repository angelegt

Dokumentation

belegt

1 h

- Quellcode hochgeladen; lokale Datenbankdatei, Backups und Zugangsdaten über .gitignore bewusst ausgeschlossen.

Aufgabe 64: Neon-Postgres-Datenbank eingerichtet und befüllt

Dokumentation

belegt

1 h

- Kostenlose Neon-Datenbank eingerichtet, 86 echte Artikel-Datensätze samt aller Stammdaten mittels Migrationsskript übertragen.

Aufgabe 65: Deployment auf Streamlit Community Cloud

Dokumentation

belegt

1 h

- Verbunden mit dem GitHub-Repository, Datenbankverbindung als geschütztes Secret hinterlegt.

Aufgabe 66: Zugriff auf eingeladene Personen beschränkt

Dokumentation

belegt

0,25 h

- „Only specific people can view this app“ aktiviert – der Link ist nicht öffentlich zugänglich.

Aufgabe 67: Live-Test im Browser gegen echte Daten

Dokumentation

belegt

1,5 h

- ___ Anzeige der 86 Artikel und 50 Artikelgruppen, Anlegen, Bearbeiten, Entnehmen, Löschen sowie Papierkorb-Funktion getestet.
- ___ Parallel verifiziert, dass der weiterhin aktive lokale Mac-Betrieb davon unberührt blieb.

Aufgabe 68: Fehler gemeldet: Auftraggeber meldet Fehler bei „Artikel Liste“/„Reste“ und „Hinzufügen“

Fehler

belegt

0,5 h

- ___ Kurz nach der ersten Veröffentlichung meldete der Auftraggeber Fehlermeldungen beim Aufrufen der Seiten „Artikel Liste“ und „Reste“ sowie beim Anlegen eines neuen Artikels über den „Hinzufügen“-Dialog.
- ___ Ursache: dieselbe Postgres-Eigenheit bei der Groß-/Kleinschreibung, diesmal an Stellen, die von den ursprünglichen Tests nicht abgedeckt waren (positionaler Listenzugriff, zufällige Alias-Namensüberschneidung).

Aufgabe 69: Korrektur: grundlegend statt symptomatisch behoben

Korrektur/Lösung

belegt

2 h

- ___ Die _Zeile-Klasse benennt Spalten seither bereits beim Erzeugen der Zeile korrekt um, statt nur die zwei gemeldeten Symptome einzeln zu flicken.
- ___ Erneut vollständig lokal nachgestellt (inklusive des exakt gemeldeten Ablaufs), vor der Veröffentlichung durch einen kompletten Klick-Test bestätigt und anschließend live verifiziert.

Aufgabe 70: Zeile-Klasse für die Dokumentation im Detail beschrieben

Zusatzaufgabe

belegt

1 h

- ___ Nachvollzogen und dokumentiert, warum der Positions- und Namenszugriff gleichzeitig korrekt bleiben muss.

Aufgabe 71: Schreib-Lock für kollisionsfreie Barcode-Vergabe dokumentiert

Zusatzaufgabe

belegt

0,75 h

- ___ „Höchste Nummer suchen“ und „neue Zeile einfügen“ laufen als ununterbrechbare Einheit innerhalb desselben threading.Lock.

Aufgabe 72: Migrationsskript wiederholbar gemacht (Code-Vertiefung)

Zusatzaufgabe

belegt

0,75 h

- ___ migrate_sqlite_to_postgres.py löscht vor jeder Tabelle deren Zieldaten, bevor die aktuellen Daten eingespielt werden – dadurch gefahrlos mehrfach ausführbar.
- ___ Nach dem Kopieren wird für jede Tabelle mit automatisch vergebenen IDs die Postgres-Sequenz auf den höchsten übertragenen Wert gesetzt.

Aufgabe 73: Betriebs-Skripte im Überblick zusammengefasst

Zusatzaufgabe

belegt

0,75 h

- ___ backup.sh/windows-setup/backup.py, server.sh/start_server.bat, start.command/start.bat, start_hidden.vbs dokumentiert.

Aufgabe 74: Fazit gezogen: fachliche, methodische und praktische Erkenntnisse

Dokumentation

belegt

1 h

- ___ Fachlich: eine Migration ist mehr als eine Übersetzung – die Materialentnahme-Logik ließ sich erst 1:1 übertragen, nachdem das VBA-Original vollständig verstanden war.
- ___ Methodisch: die Drei-Schichten-Architektur hat den Cloud-Umzug ermöglicht, ohne app.py/logic.py anzufassen – nur db.py wurde erweitert.
- ___ Praktisch: 86 reale Datensätze bei jeder Änderung mitgetestet – das machte mehrere Postgres-Fallstricke schon vor dem Go-Live sichtbar.
- ___ Bekannte Einschränkungen dokumentiert: Neon pausiert nach ca. 5 Minuten Inaktivität; Cloud-Backup läuft über Neons Point-in-Time-Recovery; lokale Netzwerkadresse nicht als feste IP hinterlegt.

Aufgabe 75: Projekt vollständig dokumentiert

Dokumentation	belegt	1 h
---------------	--------	-----

- 13 Seiten Projektdokumentation fertiggestellt, von der Excel/VBA-Analyse bis zur privat erreichbaren Cloud-Anwendung.
- Lokaler Mac-Betrieb und Cloud-Version laufen parallel, beide mit denselben 86 realen Artikel-Datensätzen.